

Appunti di Algoritmi per l'ingegneria

Giacomo Simonetto

Secondo semestre 2025-26

Sommario

Appunti del corso di Algoritmi per l'Ingegneria della facoltà di Ingegneria Informatica dell'Università di Padova.

Indice

1	Cenni di storia dell'algoritmica	3
2	Formalizzazioni dei concetti base	4
2.1	Algoritmo	4
2.2	Problema computazionale	4
2.3	Dimostrazione per induzione	6
2.4	Formule utili	7
3	Paradigma Divide and Conquer	8
3.1	Proprietà di sottostruttura	8
3.2	Analisi della correttezza	9
3.3	Albero della ricorsione	9
3.4	Analisi della complessità e formulone	10
3.5	Master theorem	11
3.6	Applicazioni del divide and conquer	13
4	Esponenziazione con esponente intero	14
4.1	Algoritmo ovvio basato sulla definizione	14
4.2	Algoritmo efficiente basato sul divide and conquer	14
5	Moltiplicazione di polinomi - Karatsuba	15
5.1	Prodotto tra polinomi usando la definizione	15
5.2	Prodotto tra polinomi con approccio divide and conquer	15
5.3	Prodotto di polinomi con l'algoritmo di Karatsuba (1962)	16
6	Moltiplicazione di matrici - Strassen	17
6.1	Prodotto di matrici usando la definizione	17
6.2	Prodotto di matrici con approccio divide and conquer	17
6.3	Prodotto di matrici con l'algoritmo di Strassen (1969)	18
6.4	Algoritmo ibrido per il prodotto di matrici	19
7	Moltiplicazione di matrici supercircolanti - Tr. di Hadamard	21
7.1	Moltiplicazioni tra matrici supercircolanti	21
7.2	Trasformata di Hadamard e Fast Hadamard Transform (FHT)	22
8	Moltiplicazione di polinomi e FFT - Fast Fourier Transform	23
9	Programmazione dinamica	24
10	Paradigma Greedy	25

1 Cenni di storia dell'algoritmica

...

2 Formalizzazioni dei concetti base

2.1 Algoritmo

Definizione formale

Un algoritmo è una macchina a stati finiti che riceve una sequenza di input, effettua una sequenza finita di transizioni tra gli stati interni della macchina e produce una sequenza di output.

Si osserva che tale definizione coincide con la definizione di una macchina di Turing.

Stopping problem

Si definisce lo “stopping problem” come la proprietà di un algoritmo di non poter continuare all’infinito. Ad esempio non esiste un algoritmo che calcoli tutte le cifre di π o che verifichi se un’equazione diofantea (o diofantina) ha soluzioni intere.

Progettazione di un algoritmo

La progettazione di un algoritmo è il processo di ideazione e formalizzazione di un algoritmo per risolvere uno specifico problema computazionale. Questo processo coinvolge:

- la conoscenza del dominio del problema ed eventuali proprietà che lo caratterizzano, in quanto potrebbero semplificare il processo di risoluzione
- la conoscenza di metodi generali di progettazione di algoritmi, come i paradigmi di programmazione

Analisi di un algoritmo

L’analisi di un algoritmo è il processo di valutazione dell’algoritmo secondo i diversi aspetti:

- correttezza: valuta se l’algoritmo porta alla soluzione corretta del problema per ogni possibile input
- efficienza: valuta il consumo di risorse in termini di passi effettuati (complessità temporale), di memoria occupata (complessità spaziale) ed eventuali altri parametri (es. consumo energetico)

2.2 Problema computazionale

Definizione formale

Un problema computazionale è composto dai seguenti tre elementi:

- un insieme $\mathcal{I}$ di istanze, che rappresentano i possibili input del problema, si nota che alcuni problemi potrebbero avere un numero infinito di istanze, per cui $\mathcal{I}$ potrebbe essere un insieme infinito
- un insieme $\mathcal{S}$ di soluzioni, che rappresentano tutti gli output del problema per ogni istanza di $\mathcal{I}$
- una relazione $\pi \subseteq \mathcal{I} \times \mathcal{S}$ che associa ogni istanza $i \in \mathcal{I}$ ad una o più soluzioni $s_j \in \mathcal{S}$ valide per quella istanza (è un sottoinsieme del prodotto cartesiano tra $\mathcal{I}$ e $\mathcal{S}$)

Rappresentazione delle istanze e delle soluzioni come stringhe

I dati di un’istanza e di una soluzione sono rappresentati come stringhe, ovvero sequenze finite di simboli. Si assumono le seguenti convenzioni:

$\Sigma = \{0, 1, \dots\}$	l’alfabeto dei simboli delle stringhe
$\Sigma^0 = \{\varepsilon\}$	l’insieme della sola stringa vuota
Σ^i	l’insieme di tutte le stringhe di lunghezza i (potenza cartesiana)
$\Sigma^+ = \bigcup_{i=1}^{\infty} \Sigma^i$	l’insieme di tutte le stringhe esclusa la stringa vuota
$\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i$	l’insieme di tutte le stringhe inclusa la stringa vuota
$\mathcal{I} \subseteq \Sigma^*, \quad \mathcal{S} \subseteq \Sigma^*$	gli insiemi di istanze e soluzioni sono sottoinsiemi di Σ^*

La taglia di una istanza è la quantità di informazione necessaria per rappresentarla ed è pari al numero di simboli che compongono la stringa che la rappresenta, o meglio la sua lunghezza. Per un numero intero n rappresentato in binario, la taglia è il numero di cifre che lo compongono $\approx \log_2 n$.

Osservazioni sulla rappresentazione dei dati nell'insieme delle istanze

L'insieme delle istanze $\mathcal{I}$ impone il tipo di rappresentazione in cui i dati in input devono essere codificati. Fornire i dati in una diversa rappresentazione potrebbe portare a soluzioni errate, ad esempio se l'algoritmo svolge la somma di numeri in base 10 e gli vengono forniti in base 2.

Inoltre la scelta della rappresentazione influisce sull'efficienza dell'algoritmo in quanto alcune operazioni risultano più efficienti in una rappresentazione rispetto ad un'altra. Ad esempio la convoluzione nel dominio del tempo è $O(n^2)$, mentre nel dominio della frequenza è $O(n \log n)$.

All'interno dell'algoritmo è possibile aggiungere una fase di conversione dei dati da una rappresentazione ad un'altra (ad esempio per ridurre la complessità computazionale), ma è sempre necessario specificare la rappresentazione dei dati in ingresso.

Funzioni e algoritmi

Il concetto di relazione è più generale di quello di funzione, in quanto una relazione può associare più elementi del codominio (es. soluzioni) ad un unico elemento del dominio (es. un'istanza). Una funzione, invece, può associare al massimo un elemento del codominio ad ogni elemento del dominio.

Un algoritmo che risolve un problema computazionale definisce una funzione tra l'insieme delle istanze e quello delle soluzioni. A partire da una istanza, restituirà soltanto una delle possibili soluzioni per tale istanza. È possibile, inoltre, che due algoritmi diversi risolvono lo stesso problema computazionale, ma producano soluzioni diverse a partire dalla stessa istanza.

Analisi della correttezza di un algoritmo

Un algoritmo $\mathcal{A}$ risolve un problema computazionale π se e solo se, detta A la funzione di trasferimento tra ingresso e uscita definita da $\mathcal{A}$, valgono le seguenti condizioni:

$$\begin{aligned} 1. \mathcal{I}_{\text{istanze}} \subseteq I_{\text{input}} \quad & 2. \mathcal{S}_{\text{soluzioni}} \subseteq O_{\text{output}} \quad & 3. \forall i \in \mathcal{I} \text{ vale } (i, A(i)) \in \pi \\ \text{con} \quad & \pi \subseteq \mathcal{I}_{\text{istanze}} \times \mathcal{S}_{\text{soluzioni}} \quad & \text{e} \quad A : I_{\text{input}} \rightarrow O_{\text{output}} \end{aligned}$$

L'analisi della correttezza consiste in particolare nel verificare la terza condizione.

Analisi della complessità di un algoritmo

L'analisi della complessità computazionale di un algoritmo consiste nel valutare il tempo necessario all'algoritmo per raggiungere la soluzione a partire da una data istanza generica.

La misurazione del tempo effettivo impiegato dipende molto dall'architettura della macchina su cui avviene la misurazione. Il valore ottenuto sarebbe troppo variabile e non permette di stimare in maniera corretta e generale la complessità di un algoritmo. Per questo motivo si simula l'esecuzione su random access machines con instruction set semplici e tempo di esecuzione costante per ogni istruzione, usato come unità di misura del tempo di esecuzione. Questa semplificazione permette sia di stimare la complessità di un algoritmo in maniera più generale, sia di confrontare tra loro algoritmi diversi prima di implementarli su una macchina reale.

Si effettua l'analisi al caso peggiore (worst-case) definendo come tempo impiegato da un algoritmo $\mathcal{A}$ per risolvere un'istanza i di taglia n come:

$$T_{\mathcal{A}}(n) := \sup_{x \in \Sigma^n} T_{\mathcal{A}}(x) \quad \text{con } \{x : |x| = n \wedge x \in \Sigma^n\}$$

L'analisi al caso migliore è poco utile, mentre l'analisi al caso medio è notevolmente più complessa da effettuare, in quanto richiede di conoscere a priori la distribuzione di probabilità delle varie istanze per una specifica taglia n . In alcuni casi quest'ultima viene preferita all'analisi al caso peggiore, se il caso peggiore è molto raro e mediamente (per la maggior parte dei casi) la complessità risulta inferiore, ad esempio per il quicksort o per l'algoritmo del semplice.

2.3 Dimostrazione per induzione

Assiomi di Peano dei numeri naturali

1. esiste un numero naturale 0
2. ogni numero naturale n ha un successore $s(n)$
3. numeri diversi hanno successori diversi: $n \neq m \implies s(n) \neq s(m)$
4. 0 non è il successore di nessun numero naturale: $s(n) \neq 0 \quad \forall n$
5. forma debole del principio di induzione:

$$\left[\underbrace{P(0)}_{\text{caso base}} \wedge \underbrace{\forall n (P(n) \implies P(n+1))}_{\text{passo induttivo}} \right] \implies P(n) \quad \forall n$$

Il quinto assioma può essere sostituito con il principio del minimo intero che sostiene che ogni insieme ordinabile non vuoto ha un elemento minimo. Il principio di induzione e quello del minimo intero sono equivalenti, siccome il primo implica il secondo e viceversa.

Forma forte del principio di induzione

Esiste una formulazione più forte del principio di induzione, equivalente alla forma debole enunciata sopra, ma che facilita le dimostrazioni basate su questo principio.

$$\left[\underbrace{P(0)}_{\text{caso base}} \wedge \underbrace{\forall n ((P(0) \wedge P(1) \wedge \dots \wedge P(n)) \implies P(n+1))}_{\text{passo induttivo}} \right] \implies P(n) \quad \forall n$$

Tale forma forte è ad esempio usata per dimostrare la correttezza del quicksort e negli ordinali transfiniti.

Rafforzamento dell'ipotesi induttiva

Alcune ipotesi induttive più “permissive” potrebbero non essere dimostrabili tramite il principio di induzione, anche se sono vere. Si riescono invece a dimostrare ipotesi più forti e ambiziose che implicano quelle più deboli. Ad esempio potrebbe non essere dimostrabile che $T(n) \leq n$, ma si riesce a dimostrare che $T(n) \leq n - 1$ che implica $T(n) \leq n$.

Questo fenomeno paradossale viene chiamato rafforzamento dell'ipotesi induttiva. Si spiega con il fatto che l'ipotesi induttiva compare sia nelle premesse che nelle conseguenze e, un suo rafforzamento, potrebbe sia rafforzare le conseguenze che rafforzare le premesse a sua volta. Alcune volte il rafforzamento delle premesse è maggiore rispetto a quello delle conseguenze e ciò permette di dimostrare queste ultime più facilmente.

Mappatura sui numeri naturali

Il principio di induzione è definito solo sui numeri naturali e può essere applicato solo su proprietà che ricevono come argomenti numeri naturali. Per dimostrare la correttezza di un algoritmo su un certo insieme di input (che non necessariamente coincide con l'insieme dei numeri naturali) è necessario effettuare una opportuna mappatura tra gli elementi di tale insieme e i numeri naturali. Spesso si clusterizza l'insieme degli input in base alla loro taglia e si mappa ogni cluster ad un numero naturale che corrisponde alla taglia degli input in quel cluster.

Ad esempio, si vorrebbe dimostrare una certa proprietà che vale per gli alberi binari completi, ma gli alberi binari completi non sono mappabili 1:1 con i numeri naturali. Si clusterizzano, quindi, in base alla loro taglia (ad esempio in base al numero di nodi interni) e si mappa ogni cluster ad un numero naturale equivalente proprio al numero di nodi interni. Si applica quindi il principio di induzione dimostrando che la proprietà vale per ogni cluster.

Limiti dell'induzione

Il principio di induzione si utilizza principalmente per dimostrare ipotesi formulate a priori. Non aiuta a formulare nuove ipotesi, che vanno pensate con l'aiuto di altre proprietà o intuizioni.

Induzione parametrica

L'induzione parametrica o induzione empirica è una tecnica che permette di individuare i parametri di una funzione generale o di un modello generale che si ritiene soddisfare al meglio i dati. Può essere usata, ad esempio, nell'addestramento di reti neurali. In pratica consiste nell'applicare il principio di induzione imponendo che la formula valga per il caso base e per il passo induttivo.

Ad esempio di vuole calcolare la formula analitica della seguente sommatoria:

$$s(n) = \sum_{k=1}^n k^2$$

Si notano le seguenti proprietà:

- $s(n) \leq n^3$ se tutti i valori della sommatoria fossero n^2
- $s(n) \geq n^3/8$ siccome almeno $n/2$ valori della sommatoria sono maggiori o uguali a $(n/2)^2$
- $s(n) \approx n^3/3 - 1/3$ se si tratta la sommatoria come un'integrale finito

per cui si ipotizza che la formula analitica sia un'equazione di terzo grado

$$s(n) = an^3 + bn^2 + cn + d$$

- caso base

si impone che valga il caso base $s(0) = 0$ da cui si ottiene $d = 0$

- passo induttivo

si impone che valga il passo induttivo $s(n+1) = s(n) + (n+1)^2$:

$$\begin{aligned} a(n+1)^3 + b(n+1)^2 + c(n+1) + d &= an^3 + bn^2 + cn + d + (n+1)^2 \\ an^3 + (3a+b)n^2 + (3a+2b+c)n + (a+b+c+d) &= an^3 + bn^2 + cn + d + n^2 + 2n + 1 \\ 3an^2 + (3a+2b)n + (a+b+c) &= n^2 + 2n + 1 \end{aligned}$$

da cui si ottengono le tre equazioni che determinano gli altri tre parametri a , b e c :

$$3a = 1 \Rightarrow a = 1/3 \qquad 3a + 2b = 2 \Rightarrow b = 1/2 \qquad a + b + c = 1 \Rightarrow c = 1/6$$

Si osserva che è possibile trovare gli stessi quattro parametri imponendo che la formula valga per quattro valori particolari di n (ad esempio $n = 0, 1, 2, 3$) e risolvendo il sistema di quattro equazioni in quattro incognite. Tale procedimento, però, senza aver osservato all'inizio che la formula è un'equazione di terzo grado, non permette di affermare che i parametri trovati valgano per ogni n . L'equazione, ad esempio, potrebbe essere di grado superiore per cui i quattro punti scelti non sono sufficienti.

2.4 Formule utili

$$a^{\log_b c} = c^{\log_b a} \qquad \text{esponenziale di un logaritmo}$$

$$\sum_{k=0}^{n-1} r^k = \frac{r^n - 1}{r - 1} \qquad \text{serie geometrica di ragione } r$$

3 Paradigma Divide and Conquer

3.1 Proprietà di sottostruttura

Idea generale

Il paradigma divide and conquer (divide et impera) è un approccio che consiste nel suddividere un'istanza di grandi dimensioni in una o più istanze di dimensioni inferiori, più semplici da risolvere, e poi combinare le soluzioni di queste istanze per ottenere la soluzione dell'istanza originale.

Proprietà di sottostruttura

Dato un problema $\pi \subseteq \mathcal{I} \times \mathcal{S}$, il paradigma si compone di tre componenti:

- $A_D : \mathcal{I} \rightarrow \mathcal{I}^*$, funzione di divide che suddivide un'istanza $i \in \mathcal{I}$ di taglia $|i| > n_0$ in una sequenza di una o più istanze più piccole $\langle i_1, i_2, \dots, i_k \rangle$ di taglia $|i_j| < |i|$ per ogni j
- $A_C : \mathcal{S}^* \rightarrow \mathcal{S}$, funzione di combine che combina la sequenza di soluzioni $\langle s_1, s_2, \dots, s_k \rangle$ associate alle istanze $\langle i_1, i_2, \dots, i_k \rangle$ nella soluzione s associata all'istanza i
- $D\&C : \mathcal{I} \rightarrow \mathcal{S}$, chiamata ricorsiva che trasforma individualmente ogni istanza i_j nella corrispondente soluzione s_j , applicando eventualmente le due funzioni A_D e A_C in modo ricorsivo

Le due funzioni A_D e A_C devono soddisfare la seguente proprietà di sottostruttura:

$$\forall i \in \mathcal{I}, |i| > n_0 \text{ vale } [D(i) = \langle i_1, \dots, i_k \rangle \wedge \forall j (i_j, s_j) \in \pi \wedge C(\langle s_1, \dots, s_k \rangle) = s \implies (i, s) \in \pi]$$

Chiamata ricorsiva (conquer) e definizione del caso base

La chiamata ricorsiva $D\&C$ è definita come segue:

- se $|i| > n_0$ allora $D\&C(i) = A_C(A_D(i))$, ovvero riapplica il modello di divide and conquer
- se $|i| \leq n_0$ allora utilizza una soluzione diretta ed immediata per risolvere i

In generale il parametro n_0 è scelto in base al problema. Può essere il minimo valore per cui è possibile applicare il modello di divide and conquer, oppure un valore sufficientemente grande sotto di cui si conosce un algoritmo più efficiente per risolvere il problema.

Pseudocodice

```
0: Algoritmo: D&C(i)
   Input:      istanza  $i \in \mathcal{I}$ 
   Output:    soluzione  $s \in \mathcal{S}$ 

1:  if  $|i| \leq n_0$  then                                ▷ caso base
2:       $s \leftarrow A_{\text{diretto}}(i)$                         ▷ soluzione diretta ed immediata
3:      return  $s$ 
4:  else                                                ▷ passo ricorsivo
5:       $\langle i_1, i_2, \dots, i_k \rangle \leftarrow A_D(i)$         ▷ divide
6:      for  $j \leftarrow 1$  to  $k$  do                        ▷ conquer
7:           $s_j \leftarrow D\&C(i_j)$ 
8:       $s \leftarrow A_C(\langle s_1, s_2, \dots, s_k \rangle)$       ▷ combine
9:      return  $s$ 
```

3.2 Analisi della correttezza

Proprietà di sottostruttura in funzione di un parametro intero

Per dimostrare la correttezza di un algoritmo divide and conquer, si utilizza la proprietà di sottostruttura e il principio di induzione in forma forte. Per prima cosa si esprime la proprietà di sottostruttura in funzione di un parametro intero, in modo da renderla più adatta alla dimostrazione induttiva, ovvero:

$$\forall i \in \mathcal{I} \underbrace{[(i, D\&C(i)) \in \pi]}_{P(i)} = \forall n \in \mathbb{N} \underbrace{[\forall i \in \mathcal{I} \text{ con } |i| = n \text{ vale } (i, D\&C(i)) \in \pi]}_{P(n)}$$

Dimostrazione con il principio di induzione in forma forte

Si dimostra, infine, che $P(n)$ è vera per ogni $n \in \mathbb{N}$ tramite il principio di induzione in forma forte:

- si dimostra che $P(n)$ è vera per ogni $n \leq n_0$ (caso base)
- si dimostra che se $P(n)$ è vera per tutti i valori $n \leq m$ allora anche $P(m+1)$ è vera (passo induttivo)

3.3 Albero della ricorsione

Albero della ricorsione

L'albero della ricorsione è una rappresentazione grafica che rappresenta le chiamate ricorsive di un algoritmo divide and conquer con le seguenti caratteristiche:

- ogni nodo rappresenta una chiamata ricorsiva $D\&C(i)$ per un'istanza $i \in \mathcal{I}$
- ogni nodo ha come figli i nodi che rappresentano le chiamate ricorsive $D\&C(i_j)$ per ogni istanza i_j ottenuta dalla procedura di divide $A_D(i)$
- la taglia dell'istanza dei nodi figli è inferiore alla taglia dell'istanza del nodo padre, in quanto la taglia dell'istanza diminuisce ad ogni chiamata ricorsiva
- ogni foglia rappresenta una chiamata ricorsiva $D\&C(i)$ per un'istanza $i \in \mathcal{I}$ di taglia $|i| \leq n_0$, ovvero un caso base

Esplorazione dell'albero della ricorsione

L'esplorazione dell'albero della ricorsione avviene secondo la visita in-order, ovvero prima si visita il figlio sinistro, poi il nodo padre, poi il figlio destro. In questo modo, dato il nodo dell'albero che si sta attualmente visitando, si è certi che tutti i nodi a sinistra di esso sono stati visitati, e i nodi a destra di esso sono ancora da visitare.

La costruzione della soluzione avviene secondo la visita post-order, ovvero prima si risolvono le istanze dei figli, poi viene costruita la soluzione per il nodo padre, secondo l'algoritmo DFS (Depth First Search).

Lo spazio occupato è proporzionale allo spazio richiesto da ciascuna chiamata ricorsiva e dal numero di chiamate ricorsive attive contemporaneamente, ovvero la profondità dell'albero della ricorsione.

Cenni sulla ricorsione nei primi linguaggi di programmazione

La memoria a disposizione sui primi computer era molto limitata e non era sufficiente per implementare uno stack che isolasse le variabili locali delle varie chiamate ricorsive attive. I primi linguaggi di programmazione, infatti, salvavano le variabili locali in posizioni fisse di memoria, comuni ad ogni chiamata ricorsiva. Per questo motivo, all'inizio la ricorsione non era supportata ed il suo utilizzo era fortemente scoraggiato. Il primo linguaggio di programmazione a supportare nativamente la ricorsione è stato il linguaggio LISP, sviluppato da John McCarthy nel 1958.

Ad oggi, la ricorsione è supportata nativamente da tutti i principali linguaggi di programmazione ed esistono ottimizzazioni a livello hardware (istruzioni macchina per la gestione dello stack) per renderla efficiente e non sprecare memoria.

3.4 Analisi della complessità e formulone

Equazione alle ricorrenze

L'equazione alle ricorrenze è un'equazione che esprime la complessità temporale di un algoritmo divide and conquer in funzione della complessità temporale delle chiamate ricorsive e delle operazioni di divide e combine. Di seguito è riportata la forma generale con le proprietà indicate sotto:

$$T(n) = \begin{cases} T_0(n) & \text{se } n \leq n_0 \quad - \quad \text{caso base} \\ s(n) \cdot T(f(n)) + \omega(n) & \text{se } n > n_0 \quad - \quad \text{passo ricorsivo} \end{cases}$$

- $T_0(n)$ è il costo dell'algoritmo diretto per risolvere un'istanza di taglia $n \leq n_0$
- $s(n)$ è il numero di chiamate ricorsive effettuate per risolvere un'istanza di taglia n e coincide con il numero di nodi figli di ogni nodo dell'albero della ricorsione
- $f(n)$ è la taglia della sottoistanza, generata dalla procedura di divide di un'istanza di taglia n , che viene passata alla chiamata ricorsiva
- $T(f(n))$ è il costo di ogni chiamata ricorsiva per risolvere un'istanza di taglia $f(n)$
- $\omega(n) = T_D(n) + T_C(n)$ è il costo complessivo delle procedure di divide e di combine

Per ottenere il valore analitico ed eliminare le ricorrenze è necessario risolvere l'equazione attraverso un'analisi analitica, o ricorrendo al formulone o al master theorem.

Iterate di f

Si definisce l'iterata di una funzione f come la potenza della composizione di f con se stessa.

$$\begin{cases} f^{(0)}(n) = n \\ f^{(l)}(n) = f(f^{(l-1)})(n) \quad \text{per } l > 0 \end{cases}$$

Si osserva che $f^{(i)}(n)$ corrisponde alla taglia dell'istanza all' i -esima chiamata ricorsiva. In particolare, inizialmente $f^{(i)}(n) = n$ per $i = 0$ e progressivamente diminuisce ad ogni iterata, fino a raggiungere il caso base $f^{(k+1)}(n) \leq n_0$ per un determinato valore di $i = k$. Si definisce k come il massimo valore di i per cui la taglia dell'istanza è ancora maggiore di n_0 , ovvero:

$$k = f^*(n, n_0) := \max \left\{ k : f^{(k)}(n) > n_0 \right\}$$

Formulone

Il formulone è una formula generale che risolve l'equazione generale alle ricorrenze indicata sopra, se sono soddisfatte opportune condizioni indicate sotto.

$$T(n) = \underbrace{\sum_{l=0}^{f^*(n, n_0)} \left[\prod_{j=0}^{l-1} s(f^{(j)}(n)) \cdot \omega(f^{(l)}(n)) \right]}_{\text{lavoro compiuto nei nodi interni}} + \underbrace{\prod_{j=0}^{f^*(n, n_0)} s(f^{(j)}(n)) \cdot T_0(f^{(f^*(n, n_0)+1)}(n))}_{\text{lavoro compiuto nelle foglie}}$$

costo interno di ogni nodo al livello l -esimo
di foglie

per ogni livello interno
nodi al livello l -esimo
costo di ogni foglia

- il numero di figli dei nodi di un certo livello è costante, ovvero $s(f^{(l)}(n)) = s_l$ per ogni livello l
- la taglia di tutti i nodi di un certo livello è costante, ovvero $f^{(l)}(n) = f_l$ per ogni livello l

Formulone applicato ad un caso particolare

Di seguito un esempio di applicazione del formulone ad un caso particolare con:

- $s(n) = a$: numero di figli costante per ogni nodo interno
- $f(n) = n/2$: la taglia dell'istanza si dimezza ad ogni chiamata ricorsiva
- $\omega(n) = \beta n$: costo di divide and conquer lineare per ogni nodo interno
- $T_0(n) = 1$: costo costante per ogni foglia

$$\begin{aligned} T(n) &= \sum_{l=0}^{(\log_2 n)-1} \left(a^l \cdot \beta \frac{n}{2^l} \right) + a^{\log_2 n} \cdot T_0(n) = \beta n \cdot \sum_{l=0}^{(\log_2 n)-1} (a/2)^l + n^{\log_2 a} = \\ &= \beta n \cdot \frac{(a/2)^{\log_2 n} - 1}{(a/2) - 1} + n^{\log_2 a} = \frac{\beta}{a/2 - 1} \cdot n \left(n^{\log_2 a/2} - 1 \right) + n^{\log_2 a} = \\ &= \underbrace{\frac{\beta}{a/2 - 1} \cdot (n^{\log_2 a} - n)}_{\text{costo dei nodi interni}} + \underbrace{n^{\log_2 a}}_{\text{costo delle foglie}} = \Theta(n^{\log_2 a}) \end{aligned}$$

Complessità degli algoritmi di ordinamento basati sui confronti

Al momento non si conoscono funzioni che legano problemi alla relativa complessità asintotica e più in generale non si conoscono metodi per individuare in maniera assoluta i lower bound di problemi, ad eccezione di quelli deducibili banalmente.

Ad esempio gli algoritmi di ordinamento basati sui confronti hanno lower bound banale di $\Theta(n \log n)$, siccome $n \log n$ è la quantità di informazione necessaria a rappresentare tutte le possibili permutazioni di n elementi ed equivale al numero minimo di confronti (rappresentabili in bit) necessari per distinguere tutte le possibili permutazioni.

$$\# \text{permutazioni} = n! \approx n^n \quad \text{informazione}(\text{perm}_i) = \log_2(n!) \approx \log_2(n^n) = n \log_2 n$$

3.5 Master theorem

Assunzioni iniziali

Il master theorem è un teorema che fornisce una soluzione asintotica immediata in funzione di due parametri a e b per una classe di equazioni alle ricorrenze con le seguenti caratteristiche:

- $s(n) = a$: numero di figli costante per ogni nodo interno
- $f(n) = n/b$: la taglia dell'istanza si divide per una costante $b > 1$ ad ogni chiamata ricorsiva
- $n_0 = 1$: caso base per istanze di taglia 1

$$T(n) = \begin{cases} T_0(n) & \text{se } n \leq 1 \\ a \cdot T(n/b) + \omega(n) & \text{se } n > 1 \end{cases}$$

Funzione soglia

Si definisce una **funzione soglia** $\sigma(n)$ che rappresenta il numero di foglie dell'albero della ricorsione. Permette di calcolare in maniera immediata il costo complessivo dovuto al lavoro compiuto nelle foglie che funge da lower bound della complessità dell'algoritmo.

$$\sigma(n) = n^{\log_b a} \quad T(n) \geq T_{\text{foglie}}(n) = T_0(n) \cdot \# \text{foglie} = T_0(n) \cdot \sigma(n)$$

Classi di funzioni e relative complessità asintotiche

Paragonando il costo dovuto al lavoro compiuto da ogni nodo interno $\omega(n)$ con la funzione soglia $\sigma(n)$, si individuano tre casi per cui è possibile calcolare la complessità asintotica di $T(n)$ in maniera immediata:

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{se il lavoro nei nodi interni } \omega(n) \text{ è trascurabile rispetto a quello delle foglie } \sigma(n) \\ \Theta(n^{\log_b a} \log_b n) & \text{se il lavoro nei nodi interni } \omega(n) \text{ è dello stesso ordine di quello delle foglie } \sigma(n) \\ \Theta(\omega(n)) & \text{se il lavoro nei nodi interni } \omega(n) \text{ domina quello delle foglie } \sigma(n) \end{cases}$$

Analisi del caso 1

Si verifica quando il lavoro nei nodi interni $\omega(n)$ è trascurabile rispetto a quello delle foglie $\sigma(n)$:

$$\exists \varepsilon > 0 \text{ tale che } \omega(n) = O(n^{(\log_b a) - \varepsilon}) = \sigma(n) \cdot O(n^{-\varepsilon})$$

Di seguito si osserva che calcolando il contributo complessivo dei nodi interni usando $\omega(n) = n^{(\log_b a) - \varepsilon}$, il risultato è comparabile con quello delle foglie senza far aumentare la complessità asintotica di $\Theta(n^{\log_b a})$.

$$\begin{aligned} T_{\text{int}}(n) &= \sum_{l=0}^{(\log_b n)-1} a^l \cdot \omega\left(\frac{n}{b^l}\right) = \sum_{l=0}^{(\log_b n)-1} a^l \cdot \left(\frac{n}{b^l}\right)^{(\log_b a) - \varepsilon} = n^{\log_b a} n^{-\varepsilon} \cdot \sum_{l=0}^{(\log_b n)-1} \frac{a^l}{b^{l(\log_b a) - l\varepsilon}} = \\ &= n^{\log_b a} n^{-\varepsilon} \cdot \sum_{l=0}^{(\log_b n)-1} \frac{a^l}{a^l \cdot b^{l\varepsilon}} = n^{\log_b a} n^{-\varepsilon} \cdot \sum_{l=0}^{(\log_b n)-1} (b^\varepsilon)^l = n^{\log_b a} n^{-\varepsilon} \cdot \frac{(b^\varepsilon)^{(\log_b n)} - 1}{b^\varepsilon - 1} = \\ &= n^{\log_b a} n^{-\varepsilon} \cdot \frac{n^\varepsilon - 1}{b^\varepsilon - 1} \approx n^{\log_b a} n^{-\varepsilon} \cdot \frac{n^\varepsilon}{b^\varepsilon - 1} = \frac{n^{\log_b a}}{b^\varepsilon - 1} = \frac{\sigma(n)}{b^\varepsilon - 1} = O(\sigma(n)) \end{aligned}$$

Si osserva che per $\varepsilon \rightarrow 0^+$ il coefficiente moltiplicativo sarebbe molto elevato, ma comunque costante, per cui asintoticamente trascurabile.

Analisi del caso 2

Si verifica quando il lavoro nei nodi interni $\omega(n)$ è dello stesso ordine di quello delle foglie $\sigma(n)$:

$$\omega(n) = \Theta(n^{\log_b a}) = \Theta(\sigma(n))$$

Di seguito si dimostra che il contributo complessivo dei nodi interni ha una complessità asintotica maggiore di quella delle foglie e domina la complessità totale dell'algoritmo, che risulta essere $\Theta(n^{\log_b a} \log_b n)$.

$$\begin{aligned} T_{\text{int}}(n) &= \sum_{l=0}^{(\log_b n)-1} \left(a^l \cdot \omega\left(\frac{n}{b^l}\right) \right) = \sum_{l=0}^{(\log_b n)-1} \left(a^l \cdot \left(\frac{n}{b^l}\right)^{\log_b a} \right) = n^{\log_b a} \cdot \sum_{l=0}^{(\log_b n)-1} \left(\frac{a^l}{b^{l(\log_b a)}} \right) \\ &= n^{\log_b a} \cdot \sum_{l=0}^{(\log_b n)-1} \frac{a^l}{a^l} = n^{\log_b a} \cdot (\log_b n) = \Theta(n^{\log_b a} \log_b n) \end{aligned}$$

Di seguito si osserva che il costo complessivo di ogni livello ha la stessa complessità asintotica di quello delle foglie, per cui è possibile calcolare la complessità totale moltiplicando la complessità di ogni livello per il numero di livelli. Gli algoritmi di questa tipologia (es. mergesort e quicksort) hanno la particolarità che il costo delle fasi di divide o combine è proporzionale alla taglia dell'istanza.

$$T_{l\text{-esimo livello}}(n) = a^l \cdot \omega\left(\frac{n}{b^l}\right) = a^l \cdot \left(\frac{n}{b^l}\right)^{\log_b a} = a^l \cdot \frac{n^{\log_b a}}{b^{l(\log_b a)}} = a^l \cdot \frac{n^{\log_b a}}{a^l} = n^{\log_b a} = \sigma(n)$$

Analisi del caso 3

Si verifica quando il lavoro nei nodi interni $\omega(n)$ domina quello delle foglie $\sigma(n)$:

$$\exists \varepsilon > 0 \text{ tale che } \omega(n) = \Omega(n^{\log_b a + \varepsilon}) = \sigma(n) \cdot \Omega(n^\varepsilon)$$

Dalla condizione precedente si può dedurre anche la seguente condizione di regolarità, ovvero che il lavoro interno complessivo nei nodi figli (al livello inferiore) è minore del lavoro interno del nodo padre.

$$\exists c \in \mathbb{R} \text{ con } 0 < c < 1 \text{ tale che } \forall n \in \mathbb{N} \text{ si ha } a \cdot \omega\left(\frac{n}{b}\right) \leq c \cdot \omega(n)$$

Per la dimostrazione del terzo caso si utilizza la condizione di regolarità appena enunciata, in quanto rende lo svolgimento più semplice e immediato. Di seguito sono riportati alcuni passaggi preliminari che verranno poi usati nella successiva dimostrazione.

$$a \cdot \omega\left(\frac{n}{b}\right) \leq c \cdot \omega(n) \Rightarrow \omega\left(\frac{n}{b}\right) \leq \frac{c}{a} \cdot \omega(n) \Rightarrow \omega\left(\frac{n}{b^l}\right) \leq \left(\frac{c}{a}\right)^l \cdot \omega(n)$$

$$T_{\text{int}}(n) = \sum_{l=0}^{(\log_b n)-1} a^l \cdot \omega\left(\frac{n}{b^l}\right) \leq \sum_{l=0}^{(\log_b n)-1} a^l \cdot \frac{c^l}{a^l} \cdot \omega(n) = \omega(n) \cdot \sum_{l=0}^{(\log_b n)-1} c^l < \omega(n) \cdot \frac{1}{1-c} = \Theta(\omega(n))$$

Dall'analisi precedente si osserva che il costo complessivo dei nodi interni è proporzionale al costo di ogni nodo interno e domina la complessità totale dell'algoritmo se il costo del singolo nodo interno è asintoticamente maggiore di quello delle foglie, ovvero se $\omega(n) = \Omega(n^{\log_b a + \varepsilon})$.

Considerazione sugli altri casi

Esistono funzioni che non possono essere classificate in nessuno dei tre casi del master theorem, ad esempio quando $\omega(n) = n^{\log_b a} \log n$. In questi casi è necessario risolvere l'equazione alle ricorrenze attraverso altri metodi risolutivi, ad esempio il formulone o l'analisi analitica.

3.6 Applicazioni del divide and conquer

Nei seguenti capitoli sono riportati alcuni esempi di applicazioni efficienti del paradigma divide and conquer:

- esponenziazione con esponente intero
- moltiplicazione di polinomi tramite l'algoritmo di Karatsuba
- moltiplicazione di matrici tramite l'algoritmo di Strassen
- moltiplicazione di matrici supercircolanti tramite la trasformata di Hadamard
- moltiplicazione di polinomi tramite la trasformata di Fourier (FFT) e l'algoritmo di Cooley-Tukey

4 Esponenziazione con esponente intero

Definizione formale del problema

Dato un elemento x proveniente da una struttura algebrica con operazione associativa (ad esempio numeri in $\mathbb{N}$, $\mathbb{R}$, $\mathbb{C}$, matrici, ...) costante, si vuole calcolare x^n con $n \in \mathbb{N}$. Siccome si tratta x come costante, si esclude che possa essere un polinomio per semplificare la risoluzione.

Osservazioni sulla taglia del problema

Essendo x costante, viene escluso dal calcolo della taglia che è determinata unicamente da n . Si osserva che la quantità di informazione necessaria a rappresentare n è $\log_2 n$, per cui la taglia del problema in funzione di cui va espressa la complessità è $\text{taglia}(n) = |n| = \log_2 n$, con $n = 2^{\text{taglia}(n)}$.

4.1 Algoritmo ovvio basato sulla definizione

L'algoritmo ovvio per calcolare x^n si basa sulla definizione ricorsiva di potenza come composizione multipla dell'operazione associativa della struttura algebrica, ovvero:

$$x^n := \begin{cases} x^{(0)} = 1 & \text{se } n = 0 \\ x^{(n-1)} \circ x & \text{se } n > 0 \end{cases} \quad - \quad \circ \text{ è l'operazione associativa della struttura algebrica}$$

La complessità equivale al costo di $n - 1$ composizioni dell'operazione associativa, per cui la complessità risulta essere esponenziale in funzione della taglia del problema, ovvero:

$$T(n) = n - 1 = 2^{\text{taglia}(n)} - 1 = \Theta\left(2^{\text{taglia}(n)}\right)$$

4.2 Algoritmo efficiente basato sul divide and conquer

Si osserva che se l'operazione è associativa, come richiesto nelle ipotesi, è possibile riarrangiare le composizioni di funzioni in modo da riutilizzare i risultati di composizioni già calcolate. Di seguito la definizione ricorsiva dell'algoritmo efficiente basato sul divide and conquer:

$$x^n := \begin{cases} x^{(0)} = 1 & \text{se } n = 0 \\ x^{(1)} = x & \text{se } n = 1 \\ x^{(n/2)} \circ x^{(n/2)} & \text{se } n > 1 \text{ è pari} \\ x^{\lfloor n/2 \rfloor} \circ x^{\lfloor n/2 \rfloor} \circ x & \text{se } n > 1 \text{ è dispari} \end{cases}$$

L'equazione alle ricorrenze per questo algoritmo al caso peggiore è la seguente, osservando che è sufficiente una chiamata ricorsiva per calcolare $x^{(n/2)}$ e, al caso peggiore, si aggiungono 2 operazioni di composizione.

$$T(n) = \begin{cases} 1 & \text{se } n = 0, 1 \\ T(n/2) + 2 & \text{se } n > 1 \end{cases}$$

Utilizzando il master theorem con $a = 1$, $b = 2$ e $\omega(n) = O(1)$, si ottiene una complessità lineare in funzione della taglia del problema, ovvero:

$$T(n) = \Theta(\log_2 n) = \Theta(\text{taglia}(n))$$

5 Moltiplicazione di polinomi - Karatsuba

Definizione formale di polinomio e degree bound

Un polinomio $P(x)$ di grado $N - 1$ (con N termini) è definito come:

$$P(x) := \sum_{i=0}^{N-1} a_i x^i \quad \text{con } a_0, a_1, \dots, a_{N-1} \text{ fissati}$$

Si osserva che un polinomio può essere visto anche come un numero espresso in base x con $0 \leq a_i < x$ secondo la notazione posizionale.

Il degree bound α è un limite superiore al grado di un polinomio, che indica l'insieme di tutti i polinomi di grado inferiore o uguale ad α , ovvero $N - 1 \leq \alpha$.

5.1 Prodotto tra polinomi usando la definizione

Tecnica risolutiva

Dati due polinomi $P(x)$ e $Q(x)$ di grado $N - 1$, il loro prodotto $P(x) \cdot Q(x)$ è definito come segue.

$$P(x) := \sum_{i=0}^{N-1} a_i x^i \quad Q(x) := \sum_{i=0}^{N-1} b_i x^i \quad P(x) \cdot Q(x) = \sum_{i=0}^{2N-2} c_i x^i$$

Analisi della complessità

Per calcolare il prodotto usando la definizione sono necessarie:

- N^2 moltiplicazioni tra i coefficienti a_i e b_j
- $(N - 1)^2 = N^2 - 1 - (2N - 2)$ addizioni tra i prodotti $a_i \cdot b_j$ per ottenere i coefficienti c_k

Complessivamente il prodotto usando la definizione ha complessità $\Theta(N^2)$.

5.2 Prodotto tra polinomi con approccio divide and conquer

Tecnica risolutiva

I polinomi $P(x)$ e $Q(x)$, di grado $N - 1$, possono essere scomposti in polinomi più semplici di grado $< N/2$ e il prodotto tra $P(x)$ e $Q(x)$ risulta essere in funzione del prodotto di questi polinomi più semplici, ovvero:

$$\begin{aligned} P(x) &= A(x) + B(x) \cdot x^{N/2} & Q(x) &= C(x) + D(x) \cdot x^{N/2} \\ P(x) \cdot Q(x) &= B(x) D(x) \cdot x^N + (A(x) D(x) + B(x) C(x)) \cdot x^{N/2} + A(x) C(x) \end{aligned}$$

Equazione alle ricorrenze

L'equazione alle ricorrenze per questo approccio è la seguente:

$$T(N) = \begin{cases} 1 & \text{se } N = 1 \\ 4 \cdot T(N/2) + \beta_{std} N & \text{se } N > 1 \end{cases}$$

- il caso base è rappresentato dal prodotto di due polinomi di grado 0 che richiede un solo prodotto
- il passo ricorsivo prevede 4 chiamate ricorsive per calcolare i quattro prodotti AC , AD , BC e BD tra polinomi di grado $< N/2$
- il costo per ogni nodo interno è dato dalle somme dei coefficienti dei termini di grado uguale, che è proporzionale alla taglia N

Analisi della complessità

Dal formulone si ottiene che la complessità rimane sempre $\Theta(N^2)$, da come evidenziato di seguito:

$$\begin{aligned}
 T(N) &= \sum_{l=0}^{(\log_2 N)-1} \left(4^l \cdot \beta_{std} \frac{N}{2^l} \right) + 4^{\log_2 N} \cdot T_0(n) = \beta_{std} N \cdot \sum_{l=0}^{(\log_2 N)-1} (2^{2l}/2^l) + N^{\log_2 4} = \\
 &= \beta_{std} N \cdot \sum_{l=0}^{(\log_2 N)-1} 2^l + N^2 = \beta_{std} N \cdot \frac{2^{\log_2 N} - 1}{2 - 1} + N^2 = \\
 &= \underbrace{\beta_{std} (N^2 - N)}_{\text{costo delle somme nei nodi interni}} + \underbrace{N^2}_{\text{costo dei prodotti nelle foglie}} = \Theta(N^2)
 \end{aligned}$$

5.3 Prodotto di polinomi con l'algoritmo di Karatsuba (1962)

Tecnica risolutiva

Karatsuba osserva che è possibile ricavare la somma dei due prodotti $AD+BC$ riutilizzando i due prodotti AC e BD già precedentemente calcolati come segue:

$$(A + B) \cdot (C + D) = AC + (AD + BC) + BD \implies AD + BC = (A + B) \cdot (C + D) - AC - BD$$

In questo modo il numero di prodotti da calcolare ricorsivamente si riduce da 4 a 3, con l'effetto di aumentare lievemente il costo della fase di divide dovuto al calcolo delle somme $(A + B)$ e $(C + D)$.

Equazione alle ricorrenze ed analisi della complessità

L'equazione alle ricorrenze per l'algoritmo di Karatsuba è la seguente:

$$T(N) = \begin{cases} 1 & \text{se } N = 1 \\ 3 \cdot T(N/2) + \beta_{kz} N & \text{se } N > 1 \end{cases}$$

Dal formulone si ottiene che la complessità è $\Theta(N^{\log_2 3}) \approx \Theta(N^{1.58})$, da come evidenziato di seguito:

$$\begin{aligned}
 T(N) &= \sum_{l=0}^{(\log_2 N)-1} (3^l \cdot \beta_{kz} N/2^l) + 3^{\log_2 N} \cdot T(1) = \beta_{kz} N \cdot \sum_{l=0}^{(\log_2 N)-1} ((3/2)^l) + N^{\log_2 3} = \\
 &= \beta_{kz} N \cdot \frac{(3/2)^{\log_2 N} - 1}{(3/2) - 1} + N^{\log_2 3} = 2\beta_{kz} N \cdot (N^{\log_2 3 - \log_2 2} - 1) + N^{\log_2 3} = \\
 &= \underbrace{2\beta_{kz} (N^{\log_2 3} - N)}_{\text{costo delle somme nei nodi interni}} + \underbrace{N^{\log_2 3}}_{\text{costo dei prodotti nelle foglie}} = \Theta(N^{\log_2 3}) \approx \Theta(N^{1.58})
 \end{aligned}$$

Considerazioni sull'algoritmo di Karatsuba

- per valori di N piccoli, l'algoritmo standard (D&C) rimane ancora più efficiente di quello di Karatsuba, in quanto il costo aggiuntivo della fase di divide è maggiore del risparmio ottenuto dalla chiamata ricorsiva in meno
- per valori di N molto grandi, è possibile migliorare ulteriormente l'efficienza asintotica nel calcolo del prodotto di polinomi utilizzando ad esempio l'algoritmo di Toom-Cook
- la vera novità dell'algoritmo di Karatsuba è stata quella di trovare il primo algoritmo sub-quadratico per risolvere un problema che tutti i più grandi matematici ritenevano impossibile da risolvere in meno di $\Theta(N^2)$

6 Moltiplicazione di matrici - Strassen

Note introduttive sulle matrici

Per la seguente sezione relativa al prodotto di matrici, si considerano solo matrici quadrate $n \times n$ ($\in \mathbb{R}^{n \times n}$) con $n = 2^k$ potenza di 2, per un certo $k \in \mathbb{N}$. In questo modo le matrici sono perfettamente scomponibili ricorsivamente in quattro sottomatrici di dimensione $n/2 \times n/2$.

Per convenzione si assume che la taglia di una matrice è pari al numero di righe (o di colonne) di cui è composta, ovvero $A \in \mathbb{R}^{n \times n} \Rightarrow |A| = n$.

6.1 Prodotto di matrici usando la definizione

Tecnica risolutiva

Date due matrici $A, B \in \mathbb{R}^{n \times n}$, con $n = 2^k$, il loro prodotto $C = A \cdot B$ è sempre una matrice $C \in \mathbb{R}^{n \times n}$ i cui elementi c_{ij} sono calcolati come segue:

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj} \quad \text{per } i, j = 1, 2, \dots, n$$

In altre parole il prodotto di matrici è una matrice che ha come elementi il prodotto interno dei vettori riga e colonna rispettivamente della prima matrice A e della seconda matrice B .

Analisi della complessità

Il prodotto interno tra un vettore riga e un vettore colonna richiede n moltiplicazioni e $n - 1$ addizioni, per un costo totale di $T(n) = 2n - 1$. Siccome vanno fatti n^2 prodotti interni, il costo totale della moltiplicazione di matrici risulta $\Theta(n^3)$:

$$T(n) = n^2(2n - 1) = 2n^3 - n^2 = \Theta(n^3)$$

6.2 Prodotto di matrici con approccio divide and conquer

Tecnica risolutiva

Le matrici $A, B \in \mathbb{R}^{n \times n}$ possono essere scomposte ciascuna in quattro sottomatrici di dimensioni $n/2 \times n/2$, identificati con $X_{11}, X_{12}, X_{21}, X_{22} \in \mathbb{R}^{n/2 \times n/2}$. Il prodotto $C = A \cdot B$ risulta essere in funzione dei prodotti tra queste sottomatrici.

$$\begin{aligned} C_{11} &= A_{11} \cdot B_{11} + A_{12} \cdot B_{21} & C_{21} &= A_{21} \cdot B_{11} + A_{22} \cdot B_{21} \\ C_{12} &= A_{11} \cdot B_{12} + A_{12} \cdot B_{22} & C_{22} &= A_{21} \cdot B_{12} + A_{22} \cdot B_{22} \end{aligned}$$

Equazione alle ricorrenze

L'equazione alle ricorrenze per questo approccio è la seguente:

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 8 \cdot T(n/2) + 4n^2 & \text{se } n > 1 \end{cases}$$

- il caso base è rappresentato dal prodotto di due matrici 1×1 che richiede un solo prodotto
- il passo ricorsivo prevede 8 chiamate ricorsive per calcolare gli otto prodotti tra le sottomatrici di dimensione $n/2 \times n/2$
- il costo per ogni nodo interno è dato dalle 4 somme dei prodotti tra sottomatrici $n/2 \times n/2$

Analisi della complessità

Dal formulone si ottiene che la complessità rimane sempre $\Theta(n^3)$, da come evidenziato di seguito:

$$\begin{aligned}
 T(n) &= \sum_{l=0}^{(\log_2 n)-1} \left(8^l \cdot \left(\frac{n}{2^l} \right)^2 \right) + 8^{\log_2 n} \cdot T_0(n) = n^2 \cdot \sum_{l=0}^{(\log_2 n)-1} 2^l + n^{\log_2 8} = n^2 \cdot \frac{2^{\log_2 n} - 1}{2 - 1} + n^3 = \\
 &= \underbrace{n^2 \cdot (n - 1)}_{\text{costo delle somme dei nodi interni}} + \underbrace{n^3}_{\text{costo dei prodotti alle foglie}} = 2n^3 - n^2 = \Theta(n^3)
 \end{aligned}$$

Vantaggi dell'algoritmo D&C ricorsivo rispetto a quello iterativo basato sulla definizione

Ad oggi le macchine moderne performano molto meglio con l'algoritmo D&C ricorsivo rispetto a quello iterativo basato sulla definizione in quanto si lavora con sottomatrici sempre più piccole fino a quando riescono ad essere totalmente memorizzate all'interno della cache. In questo modo si riducono drasticamente i tempi di accesso alla memoria principale dovuti ai continui cache miss quando si scorrono per intero righe e colonne per calcolare i prodotti interni.

6.3 Prodotto di matrici con l'algoritmo di Strassen (1969)

Tecnica risolutiva

Ispirandosi a Karatsuba, Strassen osserva che è possibile ricavare gli 8 prodotti tra le sottomatrici A_{ij} e B_{ij} riutilizzando solo 7 prodotti, calcolati su opportune somme e differenze di sottomatrici.

$$\begin{array}{lll}
 A_1 = (A_{12} - A_{22}) & B_1 = (B_{21} + B_{22}) & \\
 A_2 = (A_{11} + A_{22}) & B_2 = (B_{11} + B_{22}) & P_i = A_i \cdot B_i \quad \text{per } i = 1, 2, \dots, 7 \\
 A_3 = (A_{11} - A_{21}) & B_3 = (B_{11} + B_{12}) & C_{11} = P_1 + P_2 - P_4 + P_6 \\
 A_4 = (A_{11} + A_{12}) & B_4 = B_{22} & C_{12} = P_4 + P_5 \\
 A_5 = A_{11} & B_5 = (B_{12} - B_{22}) & C_{21} = P_6 + P_7 \\
 A_6 = A_{22} & B_6 = (B_{21} - B_{11}) & C_{22} = P_2 - P_3 + P_5 - P_7 \\
 A_7 = (A_{21} + A_{22}) & B_7 = B_{11} &
 \end{array}$$

Osservazioni sui prodotti di Strassen

- se si sviluppano i conti per le quattro espressioni di C_{ij} si può notare che si ottengono esattamente le stesse espressioni dell'approccio D&C standard
- esiste una simmetria tra le espressioni dei blocchi A_i e B_i : tra A_1 e B_5 , tra A_2 e B_2 , tra $-A_3$ e B_6 , tra A_4 e B_3 , tra A_5 e B_7 , tra A_6 e B_4 , tra A_7 e B_1

Equazione alle ricorrenze

L'equazione alle ricorrenze per l'algoritmo di Strassen è la seguente:

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 7 \cdot T(n/2) + 18 \cdot n^2/4 & \text{se } n > 1 \end{cases}$$

- il caso base è rappresentato dal prodotto di due matrici 1×1 che richiede un solo prodotto
- il passo ricorsivo prevede 7 chiamate ricorsive per calcolare i sette prodotti P_i tra i blocchi A_i e B_i di dimensione $n/2 \times n/2$
- il costo della fase di divide consiste nelle 10 somme e differenze tra sottomatrici $A_{ij}, B_{ij} \in \mathbb{R}^{n/2 \times n/2}$ per calcolare i blocchi A_i e B_i (complessivamente $10 \cdot n^2/4$)
- il costo della fase di conquer consiste nelle 8 somme/differenze tra i prodotti $P_i \in \mathbb{R}^{n/2 \times n/2}$ per calcolare i blocchi C_{ij} (complessivamente $8 \cdot n^2/4$)

Analisi della complessità

Applicando il formulone si ottiene che la complessità è $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$, da come mostrato di seguito:

$$\begin{aligned}
 T(n) &= \sum_{l=0}^{(\log_2 n)-1} \left(7^l \cdot \frac{18}{4} \cdot \left(\frac{n}{2^l} \right)^2 \right) + 7^{\log_2 n} \cdot T_0(n) = \frac{9}{2} n^2 \cdot \sum_{l=0}^{(\log_2 n)-1} ((7/4)^l) + n^{\log_2 7} = \\
 &= \frac{9}{2} n^2 \cdot \frac{(7/4)^{\log_2 n} - 1}{(7/4) - 1} + n^{\log_2 7} = \frac{9}{2} n^2 \cdot \frac{n^{\log_2(7/4)} - 1}{3/4} + n^{\log_2 7} = \\
 &= \frac{9}{2} \cdot \frac{4}{3} n^2 \cdot \left(n^{(\log_2 7)-2} - 1 \right) + n^{\log_2 7} = 6n^2 \cdot (n^{\log_2 7}/n^2 - 1) + n^{\log_2 7} \\
 &= \underbrace{6n^{\log_2 7} - 6n^2}_{\text{costo delle somme dei nodi interni}} + \underbrace{n^{\log_2 7}}_{\text{costo dei prodotti alle foglie}} = 7n^{\log_2 7} - 6n^2 = \Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})
 \end{aligned}$$

Osservazioni generali sull'algoritmo di Strassen

- una volta le moltiplicazioni di numeri erano molto costose, ora i blocchi logici sono altamente ottimizzati; inoltre molti processori dispongono dell'istruzione MADD (multiply and add) che consente di eseguire in un solo ciclo di clock una moltiplicazione seguita da un'addizione
- se si volesse eseguire il quadrato di una matrice A con l'algoritmo di Strassen, si risparmierebbero 5 somme e sottrazioni siccome $A_i = B_i \forall i$
- in generale non si può avere corrispondenza tra i blocchi A_i e B_i , perché se così fosse, l'algoritmo diventerebbe commutativo portando sicuramente ad un risultato sbagliato siccome la moltiplicazione di matrici non è commutativa
- utilizzando la definizione, la complessità asintotica del prodotto di matrici era n^Ω con $\Omega = 3$, con Strassen si è riusciti a ridurre il valore di Ω a $\log_2 7 \approx 2.81$; ad oggi si conoscono algoritmi per il prodotto di matrici con $\Omega \approx 2.37$, ma risultano poco vantaggiosi in quanto hanno costanti asintotiche molto elevate che li rendono svantaggiosi per valori di n piccoli

6.4 Algoritmo ibrido per il prodotto di matrici

Idea implementativa

Per valori di $n \leq n_0$ si osserva che l'algoritmo standard (D&C) rimane ancora più efficiente di quello di Strassen. Si vuole determinare tale valore n_0 e sviluppare un algoritmo tale che per $n > n_0$ utilizzi l'algoritmo di Strassen, mentre per $n < n_0$ utilizzi l'algoritmo standard (D&C).

Calcolo del parametro n_0

Per calcolare il parametro n_0 , si cerca il valore che minimizza la complessità dell'algoritmo di Strassen imponendo che per $n \leq n_0$ venga utilizzato l'algoritmo standard (D&C), ovvero:

- i livelli dell'albero della ricorsione arrivano a $(\log_2(n/n_0)) - 1$
- il costo delle foglie è pari alla complessità dell'algoritmo standard (D&C), ovvero $T_0(n) = 2n^3 - n^2$

$$\begin{aligned}
 T(n, n_0) &= \sum_{l=0}^{(\log_2(n/n_0))-1} \left(7^l \cdot \frac{18}{4} \cdot \left(\frac{n}{2^l} \right)^2 \right) + 7^{\log_2(n/n_0)} \cdot (2n_0^3 - n_0^2) = \dots = \\
 &= n^{\log_2 7} \cdot \underbrace{\frac{2n_0 + 5}{n_0^{(\log_2 7)-2}}}_{g(n_0)} - 6n^2
 \end{aligned}$$

Si procede a minimizzare la funzione $T(n, n_0)$ per trovare il valore ottimale di n_0 :

$$\min_{n_0} T(n, n_0) \longrightarrow \min_{n_0} g(n_0) \longrightarrow \begin{aligned} &n_{0,\min} \approx 10.477 \\ &g(n_{0,\min}) = 3.895 \end{aligned}$$

Considerazioni sul parametro n_0

Dal punto di vista teorico, si sceglie $n_0 = 8$ in quanto è la potenza di 2 più vicina a $n_{0,\min}$. Riscrivendo l'equazione alle ricorrenze e la complessità asintotica dell'algoritmo ibrido, si ottiene:

$$T(n, n_{0,\min}) = \begin{cases} 2n^3 - n^2 & \text{se } n \leq 8 \\ 7 \cdot T(n/2) + 18 n^2/4 & \text{se } n > 8 \end{cases} \quad T(n, n_{0,\min}) = \begin{cases} 2n^3 - n^2 & \text{se } n \leq 8 \\ 3.918 \cdot n^{\log_2 7} - 6n^2 & \text{se } n > 8 \end{cases}$$

Dal punto di vista pratico, non è detto che $n_0 = 8$ sia il valore ottimale, in quanto il modello teorico non tiene conto di overhead dovuti al cambio di algoritmo, alla gestione della cache e altri fattori specifici della macchina su cui viene eseguito l'algoritmo.

Ad oggi si sta sviluppando una branca di ricerca chiamata **adaptive software** che punta ad aggiustare dinamicamente i parametri degli algoritmi di algebra lineare (come n_0) attraverso analisi e test pratici effettuati sulla macchina stessa, al fine di ottenere le migliori prestazioni possibili per quella determinata macchina.

7 Moltiplicazione di matrici supercircolanti - Tr. di Hadamard

Definizione di matrici supercircolanti

Una matrice $A \in \mathbb{C}^{n \times n}$, con $n = 2^k$ si dice supercircolante se:

- per $n = 1$ (banalmente tutti i numeri sono considerati matrici supercircolanti))
- per $n > 1$ la matrice si può dividere in quattro blocchi $n/2 \times n/2$ uguali a due a due lungo la diagonale principale e la diagonale secondaria e tali blocchi sono a loro volta supercircolanti

$$A = \begin{bmatrix} A_1 & A_2 \\ A_2 & A_1 \end{bmatrix}$$

Osservazioni sulle matrici supercircolanti

- nelle matrici supercircolanti, ogni riga è ottenuta tramite uno shift circolare della riga precedente
- le righe successive alla prima sono univocamente definite a partire dalla prima riga, detta riga generatrice, tramite opportuni shift circolari
- dato che è sufficiente la prima riga per definire una matrice supercircolante, la taglia di una matrice supercircolante A è pari al numero di elementi della sua prima riga, ovvero n
- si osserva che l'insieme delle matrici supercircolanti è chiuso rispetto alla somma:

$$A + B = \begin{bmatrix} A_1 & A_2 \\ A_2 & A_1 \end{bmatrix} + \begin{bmatrix} B_1 & B_2 \\ B_2 & B_1 \end{bmatrix} = \begin{bmatrix} A_1 + B_1 & A_2 + B_2 \\ A_2 + B_2 & A_1 + B_1 \end{bmatrix} \quad \text{supercircolante per def.}$$

- si osserva che l'insieme delle matrici supercircolanti è chiuso rispetto alla moltiplicazione:

$$A \cdot B = \begin{bmatrix} A_1 & A_2 \\ A_2 & A_1 \end{bmatrix} \cdot \begin{bmatrix} B_1 & B_2 \\ B_2 & B_1 \end{bmatrix} = \begin{bmatrix} A_1 B_1 + A_2 B_2 & A_1 B_2 + A_2 B_1 \\ A_2 B_1 + A_1 B_2 & A_2 B_2 + A_1 B_1 \end{bmatrix} \quad \text{supercircolante per def.}$$

- siccome l'insieme delle matrici supercircolanti è chiuso rispetto alle operazioni di somma e moltiplicazione, allora è considerato un anello, in più si osserva che è anche commutativo, siccome le due operazioni godono della proprietà commutativa, ovvero $A + B = B + A$ e $A \cdot B = B \cdot A$

Somme tra matrici supercircolanti

Per sommare due matrici supercircolanti è sufficiente sommare le prime righe dei due addendi elemento per elemento, ottenendo la prima riga della matrice risultante dalla somma. La complessità di questo approccio è $T_{add}(n) = \Theta(n)$ con n pari alla taglia delle matrici.

7.1 Moltiplicazioni tra matrici supercircolanti

Moltiplicazioni usando la definizione

Usando la definizione di moltiplicazione di matrici, si osserva che si richiede il calcolo di 4 prodotti tra sottomatrici supercircolanti di dimensione $n/2 \times n/2$ e 2 somme tra matrici supercircolanti di dimensione $n/2 \times n/2$. L'equazione alle ricorrenze per questo approccio è la seguente:

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 4 \cdot T(n/2) + 2 \cdot n/2 & \text{se } n > 1 \end{cases} = \begin{cases} 1 & \text{se } n = 1 \\ 4 \cdot T(n/2) + n & \text{se } n > 1 \end{cases}$$

Dal master theorem con $a = 4$, $b = 2$ e $\omega(n) = n$ si ottiene una funzione soglia di $\sigma(n) = n^{\log_2 4} = n^2$, tale per cui $\omega(n) = \sigma(n) \cdot O(n^{-1})$, ovvero si ricade nel primo caso del master theorem. La complessità è quindi $T(n) = \Theta(\sigma(n)) = \Theta(n^2)$.

Moltiplicazioni con approccio D&C ottimizzato

Si osserva che secondo opportune manipolazioni algebriche è possibile ricavare i due blocchi C_1 e C_2 del prodotto attraverso soli 2 prodotti, con l'overhead di 6 somme e due moltiplicazioni per uno scalare.

$$\begin{aligned} U &= (A_1 + A_2) \cdot (B_1 + B_2) = A_1 B_1 + A_1 B_2 + A_2 B_1 + A_2 B_2 & C_1 &= (U + V)/2 = A_1 B_1 + A_2 B_2 \\ V &= (A_1 - A_2) \cdot (B_1 - B_2) = A_1 B_1 - A_1 B_2 - A_2 B_1 + A_2 B_2 & C_2 &= (U - V)/2 = A_1 B_2 + A_2 B_1 \end{aligned}$$

L'equazione alle ricorrenze per questo approccio è la seguente:

$$T(n) = \begin{cases} 1 & \text{se } n = 1 \\ 2 \cdot T(n/2) + 6 \cdot n/2 + 2 \cdot n/2 & \text{se } n > 1 \end{cases} = \begin{cases} 1 & \text{se } n = 1 \\ 2 \cdot T(n/2) + 4n & \text{se } n > 1 \end{cases}$$

Dal master theorem con $a = 2$, $b = 2$ e $\omega(n) = 4n$ si ottiene una funzione soglia di $\sigma(n) = n^{\log_2 2} = n$ paragonabile a $\omega(n)$. La complessità dal secondo caso del master theorem risulta essere $T(n) = \Theta(n \cdot \log n)$.

7.2 Trasformata di Hadamard e Fast Hadamard Transform (FHT)

Autovalori alle foglie dell'albero della ricorsione

L'albero della ricorsione per il prodotto di matrici supercircolanti con approccio D&C ottimizzato è un albero binario completo in cui ogni nodo figlio sinistro riceve le somme tra i blocchi A_1 e A_2 e tra i blocchi B_1 e B_2 , mentre ogni nodo figlio destro riceve le differenze tra i medesimi blocchi, affinché siano moltiplicate opportunamente.

Le foglie ricevono due matrici degeneri in scalari a e b che è possibile dimostrare essere proprio gli autovalori delle due matrici supercircolanti A e B di partenza.

Trasformata e prodotto di Hadamard

È possibile interpretare le operazioni di divide come una trasformazione lineare che, partendo da una matrice supercircolante rappresentata con la sua prima riga, restituisce il vettore di autovalori della matrice supercircolante stessa. In questo modo il prodotto di matrici supercircolanti può essere interpretato come un prodotto di autovalori, che risulta avere complessità $\Theta(n)$ con n pari alla taglia delle matrici supercircolanti. Una volta fatto ciò, le operazioni di combine effettuano l'inverso della trasformazione lineare, restituendo la matrice supercircolante risultante dal prodotto.

La trasformazione lineare impiegata prende il nome di **trasformata di Hadamard**, mentre il prodotto membro a membro tra i vettori di autovalori prende il nome di **prodotto di Hadamard**.

La trasformata e la relativa antitrasformata di Hadamard possono essere calcolate in $\Theta(n \cdot \log n)$, mentre il prodotto di Hadamard può essere calcolato in $\Theta(n)$. Di conseguenza, il prodotto di matrici supercircolanti tramite la trasformata di Hadamard ha complessità complessiva di $\Theta(n \cdot \log n)$.

Cambio di rappresentazioni per risolvere problemi più efficientemente

L'idea alla base del prodotto di matrici supercircolanti tramite la trasformata di Hadamard è quello di convertire i dati in ingresso in una rappresentazione che permette di risolvere il problema richiesto in maniera più efficiente, per poi convertire il risultato ottenuto nella rappresentazione originale. Altri esempi di algoritmi che adottano questa tecnica sono:

- l'algoritmo di Strassen, anche se non è chiaro come interpretare la rappresentazione intermedia
- l'algoritmo di FFT, che utilizza una trasformata di Fourier per calcolare il prodotto di polinomi in maniera più efficiente

Osservazione sulle simmetrie

Una simmetria è una proprietà di invarianza rispetto ad una certa trasformazione. Ad esempio la proprietà commutativa è l'invarianza rispetto alla trasformazione che scambia i due operandi di un'operazione.

Conoscere le simmetrie permette di identificare ed eliminare i gradi di libertà di un problema, riducendo sia il peso della rappresentazione utilizzata che nelle operazioni di calcolo.

8 Moltiplicazione di polinomi e FFT - Fast Fourier Transform

9 Programmazione dinamica

10 Paradigma Greedy